

MES Abasaheb Garware College (Autonomous), Pune

S.Y.B.Sc. (Data Science) – Semester III
DTS-202-M: Database Management System

ASSIGNMENT NO. 12

MINI PROJECT – TRAINING INSTITUTE MANAGEMENT SYSTEM

Academic Year: 2026–2027

Student Name	Amit Panorekar	Roll Number	20968
Batch	SY. BSc Data Science	Topic	WB(Assignment-12)

1. Objective

Develop a basic MySQL Training Institute Management System to manage students, trainers, courses, batches, enrollments, attendance, and assessment results. The project demonstrates database planning, table relationships, query writing, business analysis, and presentation of findings.

2. Requirement Analysis and Project Planning

- Study how a training institute manages students, trainers, courses, batches, attendance, and assessment results.
- Identify the main users: administrator, trainer, and institute coordinator.
- Define the information stored for each business activity.
- Prepare functional requirements and identify the main reports expected from the database.
- Use realistic sample data representing a training institute.

3. Functional Requirements

- Maintain student registration and contact information.
- Maintain course details such as duration, fee, and level.
- Maintain trainer details, specialization, and experience.
- Create batches for courses and assign trainers.
- Enroll students into batches.
- Record attendance for each enrolled student and training session.
- Record assessment results for enrolled students.
- Generate reports for enrollment, attendance, performance, and training progress.

4. ER Diagram

The ER diagram follows the seven-table structure and relationship requirements specified in the supplied Assignment No. 12. It uses the same textbook-style notation as the supplied reference image.

ER DIAGRAM – TRAINING INSTITUTE MANAGEMENT SYSTEM

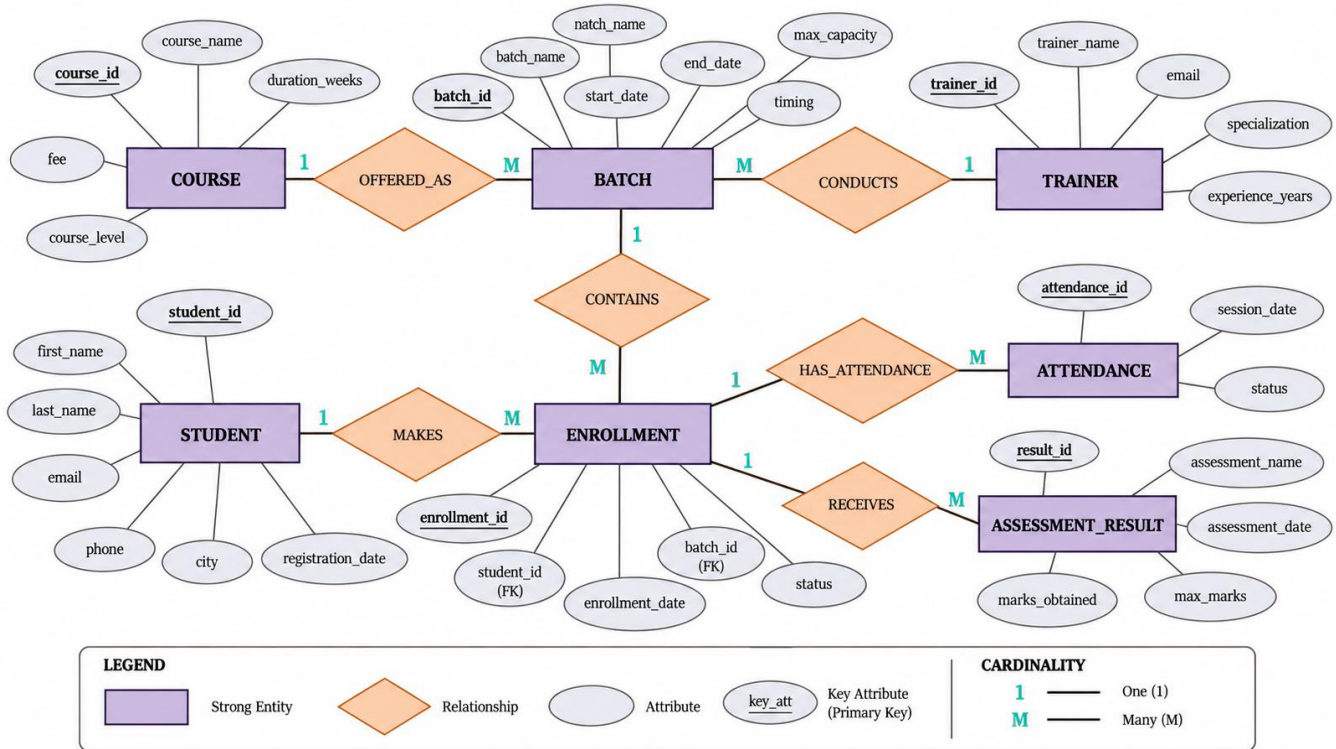


Figure 1: ER Diagram – Training Institute Management System

5. Database Schema Diagram

The table schema below follows the same schema-diagram approach shown in the supplied reference image. It represents the seven required entities in the Training Institute Management System and shows each column, data type, primary key, and foreign-key relationship.

TRAINING INSTITUTE MANAGEMENT SYSTEM TABLE SCHEMA DIAGRAM

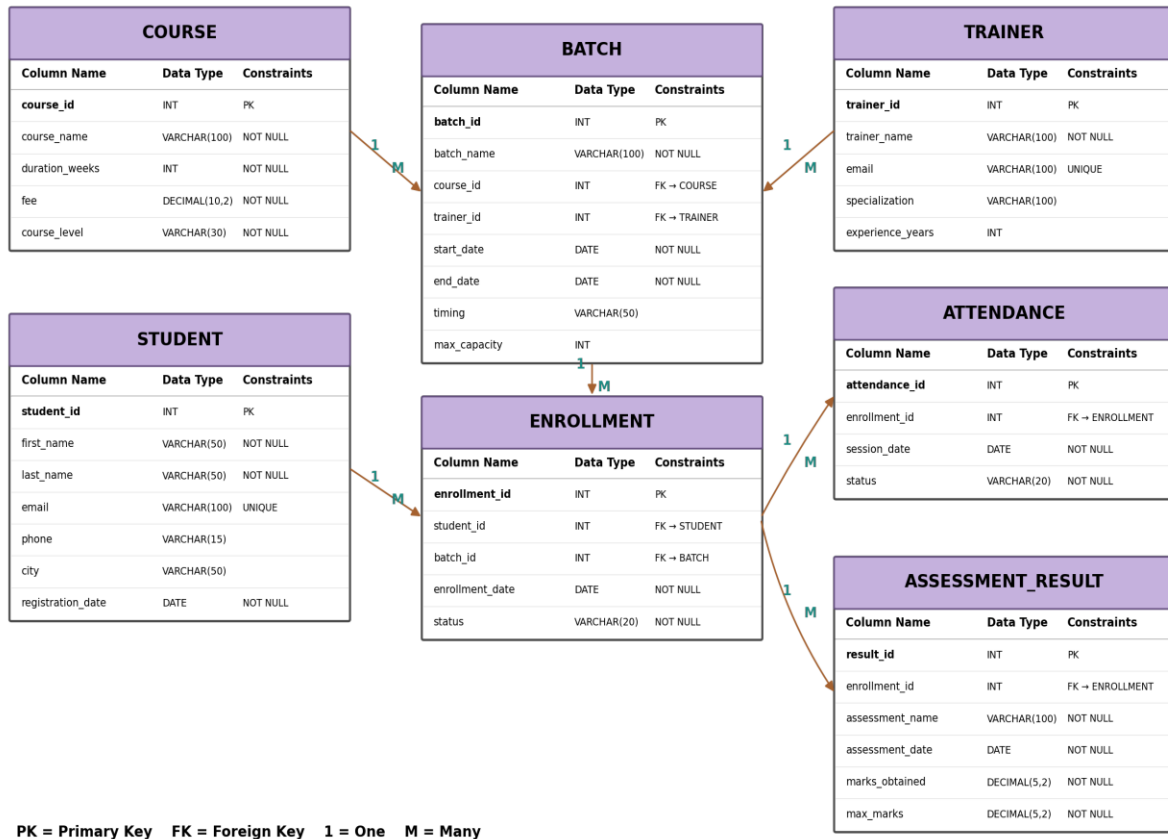


Figure 2: Table Schema Diagram – Training Institute Management System

Schema interpretation: Course and Trainer are parent tables for Batch. Batch connects the selected course and trainer. Enrollment connects Student with Batch. Attendance and AssessmentResult store repeated records for each enrollment. Therefore, the foreign keys shown in the schema directly implement the relationships shown in the ER diagram.

5. Entities and Attributes

Entity	Attributes
Student	student_id (PK), first_name, last_name, email, phone, city, registration_date
Trainer	trainer_id (PK), trainer_name, email, specialization, experience_years
Course	course_id (PK), course_name, duration_weeks, fee, course_level
Batch	batch_id (PK), batch_name, course_id (FK), trainer_id (FK), start_date, end_date, timing, max_capacity
Enrollment	enrollment_id (PK), student_id (FK), batch_id (FK), enrollment_date, status
Attendance	attendance_id (PK), enrollment_id (FK), session_date, status
AssessmentResult	result_id (PK), enrollment_id (FK), assessment_name, assessment_date, marks_obtained, max_marks

6. Table Relationships and Cardinality

Parent Table	Related Table	Relationship	Purpose
Course	Batch	1 : M	One course can be conducted through multiple batches.
Trainer	Batch	1 : M	One trainer can handle multiple batches.
Student	Enrollment	1 : M	One student can have multiple enrollments.
Batch	Enrollment	1 : M	One batch can contain multiple student enrollments.
Enrollment	Attendance	1 : M	One enrollment can have multiple attendance records.
Enrollment	AssessmentResult	1 : M	One enrolled student can receive multiple assessment results.

7. Primary Keys and Foreign Keys

Table	Primary Key	Foreign Key(s)
Course	course_id	—
Trainer	trainer_id	—
Batch	batch_id	course_id → Course.course_id; trainer_id → Trainer.trainer_id
Student	student_id	—
Enrollment	enrollment_id	student_id → Student.student_id; batch_id → Batch.batch_id
Attendance	attendance_id	enrollment_id → Enrollment.enrollment_id
AssessmentResult	result_id	enrollment_id → Enrollment.enrollment_id

8. Relational Schema / Table Structure

Student

Column	Data Type	Constraint
student_id	INT	PRIMARY KEY
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
phone	VARCHAR(15)	
city	VARCHAR(50)	
registration_date	DATE	NOT NULL

Course

Column	Data Type	Constraint
course_id	INT	PRIMARY KEY
course_name	VARCHAR(100)	NOT NULL
duration_weeks	INT	NOT NULL
fee	DECIMAL(10,2)	NOT NULL
course_level	VARCHAR(30)	NOT NULL

Trainer

Column	Data Type	Constraint
trainer_id	INT	PRIMARY KEY
trainer_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
specialization	VARCHAR(100)	

experience_years	INT	
------------------	-----	--

Batch

Column	Data Type	Constraint
batch_id	INT	PRIMARY KEY
batch_name	VARCHAR(100)	NOT NULL
course_id	INT	NOT NULL, FK
trainer_id	INT	NOT NULL, FK
start_date	DATE	NOT NULL
end_date	DATE	NOT NULL
timing	VARCHAR(50)	
max_capacity	INT	

Enrollment

Column	Data Type	Constraint
enrollment_id	INT	PRIMARY KEY
student_id	INT	NOT NULL, FK
batch_id	INT	NOT NULL, FK
enrollment_date	DATE	NOT NULL
status	VARCHAR(20)	NOT NULL

Attendance

Column	Data Type	Constraint
attendance_id	INT	PRIMARY KEY
enrollment_id	INT	NOT NULL, FK
session_date	DATE	NOT NULL
status	VARCHAR(20)	NOT NULL

AssessmentResult

Column	Data Type	Constraint
result_id	INT	PRIMARY KEY
enrollment_id	INT	NOT NULL, FK
assessment_name	VARCHAR(100)	NOT NULL
assessment_date	DATE	NOT NULL
marks_obtained	DECIMAL(5,2)	NOT NULL
max_marks	DECIMAL(5,2)	NOT NULL

9. MySQL Database Creation and Table Creation

This script creates the database and the seven tables shown in the ER diagram.

```
CREATE DATABASE IF NOT EXISTS TrainingInstituteDB;

USE TrainingInstituteDB;

CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15),
    city VARCHAR(50),
```

```

        registration_date DATE NOT NULL
    );

CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    duration_weeks INT NOT NULL,
    fee DECIMAL(10,2) NOT NULL,
    course_level VARCHAR(30) NOT NULL
);

CREATE TABLE Trainer (
    trainer_id INT PRIMARY KEY,
    trainer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    specialization VARCHAR(100),
    experience_years INT
);

CREATE TABLE Batch (
    batch_id INT PRIMARY KEY,
    batch_name VARCHAR(100) NOT NULL,
    course_id INT NOT NULL,
    trainer_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    timing VARCHAR(50),
    max_capacity INT,
    FOREIGN KEY (course_id) REFERENCES Course(course_id),
    FOREIGN KEY (trainer_id) REFERENCES Trainer(trainer_id)
);

CREATE TABLE Enrollment (

```

```
    enrollment_id INT PRIMARY KEY,

    student_id INT NOT NULL,

    batch_id INT NOT NULL,

    enrollment_date DATE NOT NULL,

    status VARCHAR(20) NOT NULL,

    FOREIGN KEY (student_id) REFERENCES Student(student_id),

    FOREIGN KEY (batch_id) REFERENCES Batch(batch_id)

);


CREATE TABLE Attendance (

    attendance_id INT PRIMARY KEY,

    enrollment_id INT NOT NULL,

    session_date DATE NOT NULL,

    status VARCHAR(20) NOT NULL,

    FOREIGN KEY (enrollment_id) REFERENCES Enrollment(enrollment_id)

);


CREATE TABLE AssessmentResult (

    result_id INT PRIMARY KEY,

    enrollment_id INT NOT NULL,

    assessment_name VARCHAR(100) NOT NULL,

    assessment_date DATE NOT NULL,

    marks_obtained DECIMAL(5,2) NOT NULL,

    max_marks DECIMAL(5,2) NOT NULL,

    FOREIGN KEY (enrollment_id) REFERENCES Enrollment(enrollment_id)
```

```
);

localhost / 127.0.0.1 | phpMyAdmin
localhost/phpmyadmin/index.php?route=/server/sql

Server: 127.0.0.1
Databases SQL Status User accounts Export Import Settings Replication Variables Charsets Engines Plug-ins

Recent Favourites

New
citycare
collegedb
employeeedb
hospitaldb
information_schema
juniorcollegedb
minicollegedb
mysql
performance_schema
phpmyadmin
robotech
schooldb
seniorcollegedb
test
traininginstitutedb

Run SQL query/queries on server "127.0.0.1":

1 CREATE DATABASE IF NOT EXISTS TrainingInstituteDB;
2 USE TrainingInstituteDB;
3 CREATE TABLE Student (
4     student_id INT PRIMARY KEY,
5     first_name VARCHAR(50) NOT NULL,
6     last_name VARCHAR(50) NOT NULL,
7     email VARCHAR(100) NOT NULL UNIQUE,
8     phone VARCHAR(15),
9     city VARCHAR(50),
10    registration_date DATE NOT NULL
11);
12
13 CREATE TABLE Course (
14     course_id INT PRIMARY KEY,
15     course_name VARCHAR(100) NOT NULL,
16     duration_weeks INT NOT NULL,
17     fee DECIMAL(10,2) NOT NULL,
18     course_level VARCHAR(30) NOT NULL
19);
20
21 CREATE TABLE Trainer (
22     trainer_id INT PRIMARY KEY,
23     trainer_name VARCHAR(100) NOT NULL,
24     email VARCHAR(100) NOT NULL UNIQUE,
25     specialization VARCHAR(100),
26     experience_years INT
27);
28
29 CREATE TABLE Batch (
30     batch_id INT PRIMARY KEY,
31     batch_name VARCHAR(100) NOT NULL,
32     course_id INT NOT NULL,
```

```
localhost / 127.0.0.1 | phpMyAdmin
localhost/phpmyadmin/index.php?route=/server/sql

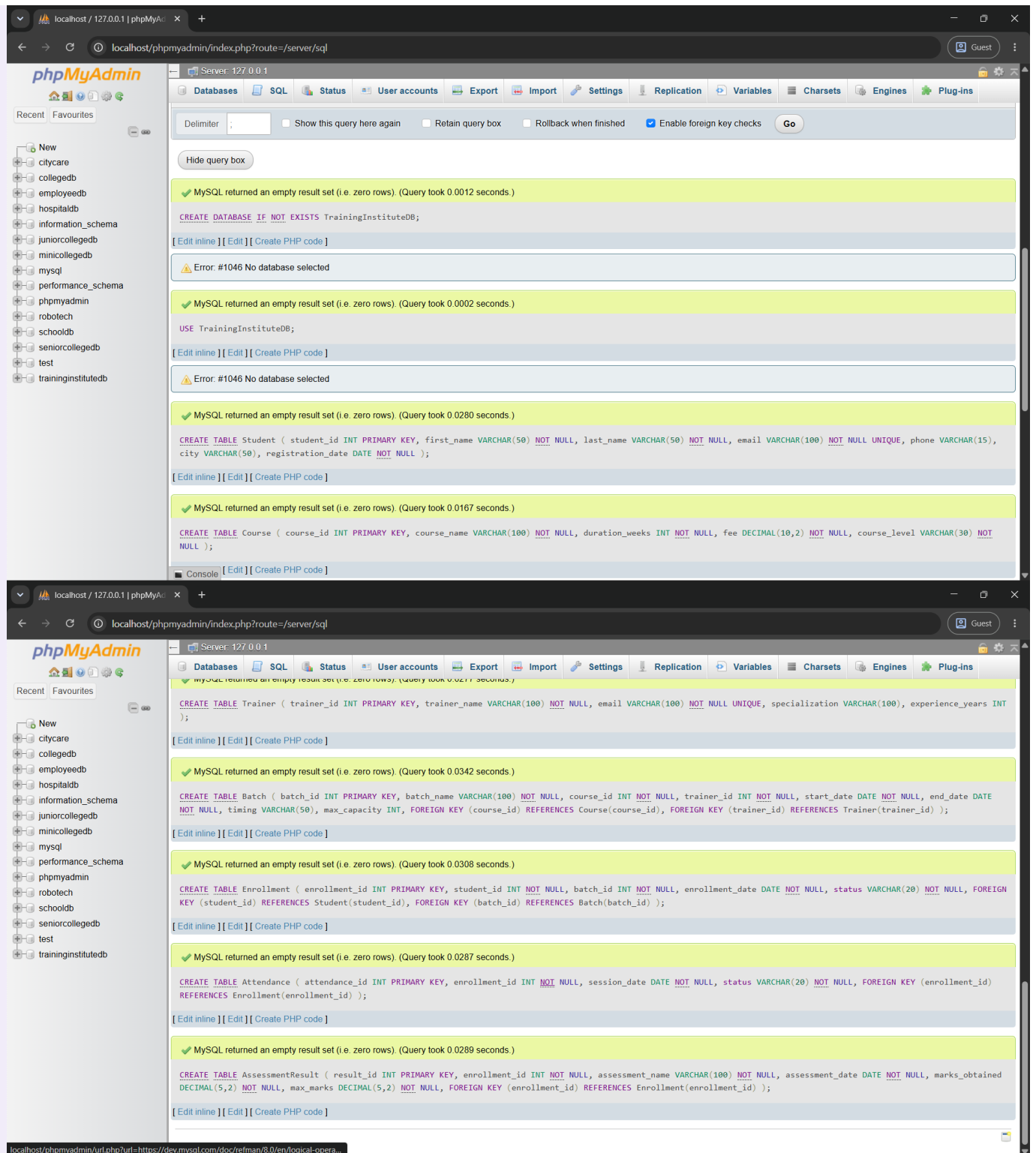
Server: 127.0.0.1
Databases SQL Status User accounts Export Import Settings Replication Variables Charsets Engines Plug-ins

Recent Favourites

New
citycare
collegedb
employeeedb
hospitaldb
information_schema
juniorcollegedb
minicollegedb
mysql
performance_schema
phpmyadmin
robotech
schooldb
seniorcollegedb
test
traininginstitutedb

Run SQL query/queries on server "127.0.0.1":

39 FOREIGN KEY (course_id) REFERENCES Course(course_id);
40 FOREIGN KEY (trainer_id) REFERENCES Trainer(trainer_id)
41);
42 CREATE TABLE Enrollment (
43     enrollment_id INT PRIMARY KEY,
44     student_id INT NOT NULL,
45     batch_id INT NOT NULL,
46     enrollment_date DATE NOT NULL,
47     status VARCHAR(20) NOT NULL,
48     FOREIGN KEY (student_id) REFERENCES Student(student_id),
49     FOREIGN KEY (batch_id) REFERENCES Batch(batch_id)
50);
51
52 CREATE TABLE Attendance (
53     attendance_id INT PRIMARY KEY,
54     enrollment_id INT NOT NULL,
55     session_date DATE NOT NULL,
56     status VARCHAR(20) NOT NULL,
57     FOREIGN KEY (enrollment_id) REFERENCES Enrollment(enrollment_id)
58);
59
60 CREATE TABLE AssessmentResult (
61     result_id INT PRIMARY KEY,
62     enrollment_id INT NOT NULL,
63     assessment_name VARCHAR(100) NOT NULL,
64     assessment_date DATE NOT NULL,
65     marks_obtained DECIMAL(5,2) NOT NULL,
66     max_marks DECIMAL(5,2) NOT NULL,
67     FOREIGN KEY (enrollment_id) REFERENCES Enrollment(enrollment_id)
68);
```



INSERT INTO Course (course_id, course_name, duration_weeks, fee, course_level) VALUES

(101,'Python Programming',8,12000.00,'Beginner'),

(102,'Data Analytics with SQL',10,15000.00,'Intermediate'),

(103,'Web Development',12,18000.00,'Intermediate'),

(104,'Data Science',16,25000.00,'Advanced');

```
INSERT INTO Trainer (trainer_id, trainer_name, email, specialization, experience_years) VALUES
```

```
(201,'Amit Kulkarni','amit.kulkarni@institute.com','Python & Data Science',7),
```

```
(202,'Neha Joshi','neha.joshi@institute.com','SQL & Database',6),
```

```
(203,'Rohan Patil','rohan.patil@institute.com','Web Development',8),
```

```
(204,'Priya Deshmukh','priya.deshmukh@institute.com','Statistics & Data Science',5);
```

```
INSERT INTO Batch (batch_id, batch_name, course_id, trainer_id, start_date, end_date, timing, max_capacity) VALUES
```

```
(301,'PY-JAN-26',101,201,'2026-01-05','2026-03-01','10:00 AM - 12:00 PM',30),
```

```
(302,'SQL-MAR-26',102,202,'2026-03-02','2026-05-10','02:00 PM - 04:00 PM',25),
```

```
(303,'WEB-APR-26',103,203,'2026-04-06','2026-06-28','11:00 AM - 01:00 PM',30),
```

```
(304,'DS-JUL-26',104,204,'2026-07-06','2026-10-25','09:00 AM - 12:00 PM',20),
```

```
(305,'PY-AUG-26',101,201,'2026-08-03','2026-09-27','04:00 PM - 06:00 PM',30);
```

```
INSERT INTO Student (student_id, first_name, last_name, email, phone, city, registration_date) VALUES
```

```
(1,'Aarav','Shah','aarav.shah@example.com','9820011101','Mumbai','2025-12-20'),
```

```
(2,'Meera','Joshi','meera.joshi@example.com','9820011102','Pune','2025-12-22'),
```

```
(3,'Rohan','Patil','rohan.patil@example.com','9820011103','Nashik','2026-02-15'),
```

```
(4,'Sneha','Kulkarni','sneha.kulkarni@example.com','9820011104','Pune','2026-02-18'),
```

```
(5,'Kunal','Mehta','kunal.mehta@example.com','9820011105','Thane','2026-02-20'),
```

```
(6,'Ananya','Sharma','ananya.sharma@example.com','9820011106','Mumbai','2026-03-25'),
```

```
(7,'Vikram','Desai','vikram.desai@example.com','9820011107','Pune','2026-03-28'),
```

```
(8,'Isha','Nair','isha.nair@example.com','9820011108','Nashik','2026-06-20'),
```

```
(9,'Aditya','More','aditya.more@example.com','9820011109','Pune','2026-06-22'),
```

```
(10,'Kavya','Verma','kavya.verma@example.com','9820011110','Mumbai','2026-06-25');
```

```
INSERT INTO Enrollment (enrollment_id, student_id, batch_id, enrollment_date, status) VALUES
```

```
(401,1,301,'2025-12-28','Completed'),
```

```
(402,2,301,'2025-12-30','Completed'),  
(403,3,302,'2026-02-25','Completed'),  
(404,4,302,'2026-02-26','Completed'),  
(405,5,302,'2026-02-27','Completed'),  
(406,6,303,'2026-03-30','Completed'),  
(407,7,303,'2026-03-31','Completed'),  
(408,8,304,'2026-06-28','Active'),  
(409,9,304,'2026-06-29','Active'),  
(410,10,304,'2026-06-30','Active'),  
(411,2,305,'2026-07-28','Completed'),  
(412,5,305,'2026-07-29','Dropped');
```

```
INSERT INTO Attendance (attendance_id, enrollment_id, session_date, status) VALUES
```

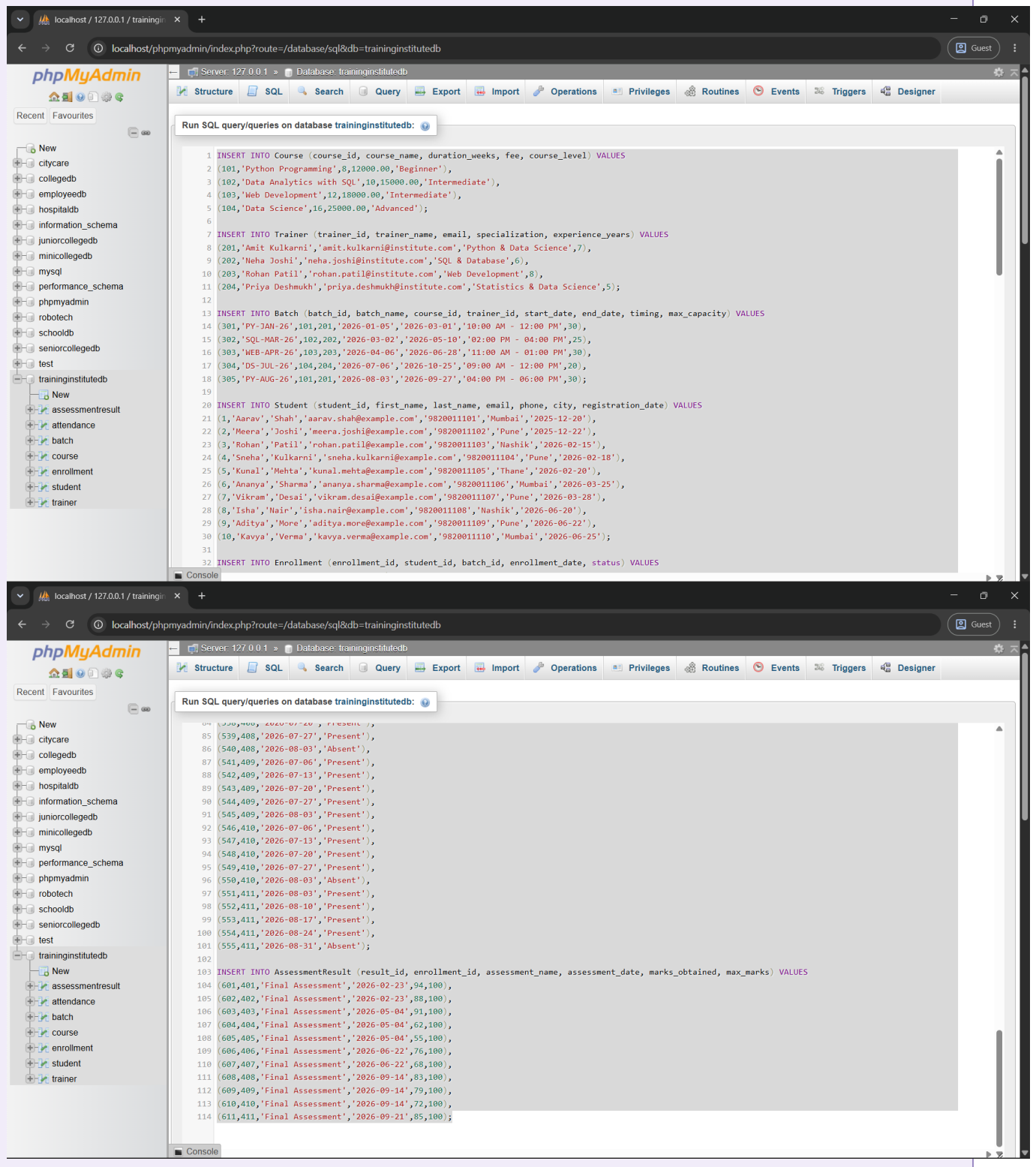
```
(501,401,'2026-01-05','Present'),  
(502,401,'2026-01-12','Present'),  
(503,401,'2026-01-19','Present'),  
(504,401,'2026-01-26','Present'),  
(505,401,'2026-02-02','Present'),  
(506,402,'2026-01-05','Present'),  
(507,402,'2026-01-12','Present'),  
(508,402,'2026-01-19','Present'),  
(509,402,'2026-01-26','Present'),  
(510,402,'2026-02-02','Absent'),  
(511,403,'2026-03-02','Present'),  
(512,403,'2026-03-09','Present'),  
(513,403,'2026-03-16','Present'),  
(514,403,'2026-03-23','Present'),  
(515,403,'2026-03-30','Absent'),
```

(516,404,'2026-03-02','Present'),
(517,404,'2026-03-09','Present'),
(518,404,'2026-03-16','Present'),
(519,404,'2026-03-23','Absent'),
(520,404,'2026-03-30','Absent'),
(521,405,'2026-03-02','Present'),
(522,405,'2026-03-09','Present'),
(523,405,'2026-03-16','Absent'),
(524,405,'2026-03-23','Absent'),
(525,405,'2026-03-30','Absent'),
(526,406,'2026-04-06','Present'),
(527,406,'2026-04-13','Present'),
(528,406,'2026-04-20','Present'),
(529,406,'2026-04-27','Present'),
(530,406,'2026-05-04','Present'),
(531,407,'2026-04-06','Present'),
(532,407,'2026-04-13','Present'),
(533,407,'2026-04-20','Present'),
(534,407,'2026-04-27','Absent'),
(535,407,'2026-05-04','Absent'),
(536,408,'2026-07-06','Present'),
(537,408,'2026-07-13','Present'),
(538,408,'2026-07-20','Present'),
(539,408,'2026-07-27','Present'),
(540,408,'2026-08-03','Absent'),
(541,409,'2026-07-06','Present'),
(542,409,'2026-07-13','Present'),
(543,409,'2026-07-20','Present'),

```
(544,409,'2026-07-27','Present'),  
(545,409,'2026-08-03','Present'),  
(546,410,'2026-07-06','Present'),  
(547,410,'2026-07-13','Present'),  
(548,410,'2026-07-20','Present'),  
(549,410,'2026-07-27','Present'),  
(550,410,'2026-08-03','Absent'),  
(551,411,'2026-08-03','Present'),  
(552,411,'2026-08-10','Present'),  
(553,411,'2026-08-17','Present'),  
(554,411,'2026-08-24','Present'),  
(555,411,'2026-08-31','Absent');
```

```
INSERT INTO AssessmentResult (result_id, enrollment_id, assessment_name, assessment_date,  
marks_obtained, max_marks) VALUES
```

```
(601,401,'Final Assessment','2026-02-23',94,100),  
(602,402,'Final Assessment','2026-02-23',88,100),  
(603,403,'Final Assessment','2026-05-04',91,100),  
(604,404,'Final Assessment','2026-05-04',62,100),  
(605,405,'Final Assessment','2026-05-04',55,100),  
(606,406,'Final Assessment','2026-06-22',76,100),  
(607,407,'Final Assessment','2026-06-22',68,100),  
(608,408,'Final Assessment','2026-09-14',83,100),  
(609,409,'Final Assessment','2026-09-14',79,100),  
(610,410,'Final Assessment','2026-09-14',72,100),  
(611,411,'Final Assessment','2026-09-21',85,100);
```



10. Sample Data

The project uses realistic sample records so that the required business questions can be tested.

Course ID	Course Name	Duration (Weeks)	Fee	Level
101	Python Programming	8	12000.00	Beginner

102	Data Analytics with SQL	10	15000.00	Intermediate
103	Web Development	12	18000.00	Intermediate
104	Data Science	16	25000.00	Advanced
Trainer ID	Trainer Name	Specialization	Experience (Years)	
201	Amit Kulkarni	Python & Data Science	7	
202	Neha Joshi	SQL & Database	6	
203	Rohan Patil	Web Development	8	
204	Priya Deshmukh	Statistics & Data Science	5	
Batch ID	Batch Name	Course ID	Trainer ID	Start Date
301	PY-JAN-26	101	201	2026-01-05
302	SQL-MAR-26	102	202	2026-03-02
303	WEB-APR-26	103	203	2026-04-06
304	DS-JUL-26	104	204	2026-07-06
305	PY-AUG-26	101	201	2026-08-03
Student ID	Student Name	City		
1	Aarav Shah	Mumbai		
2	Meera Joshi	Pune		
3	Rohan Patil	Nashik		
4	Sneha Kulkarni	Pune		
5	Kunal Mehta	Thane		
6	Ananya Sharma	Mumbai		
7	Vikram Desai	Pune		
8	Isha Nair	Nashik		
9	Aditya More	Pune		
10	Kavya Verma	Mumbai		
Enrollment ID	Student ID	Batch ID		
401	1	301		
402	2	301		
403	3	302		
404	4	302		
405	5	302		
406	6	303		
407	7	303		
408	8	304		
409	9	304		
410	10	304		
411	2	305		
412	5	305		

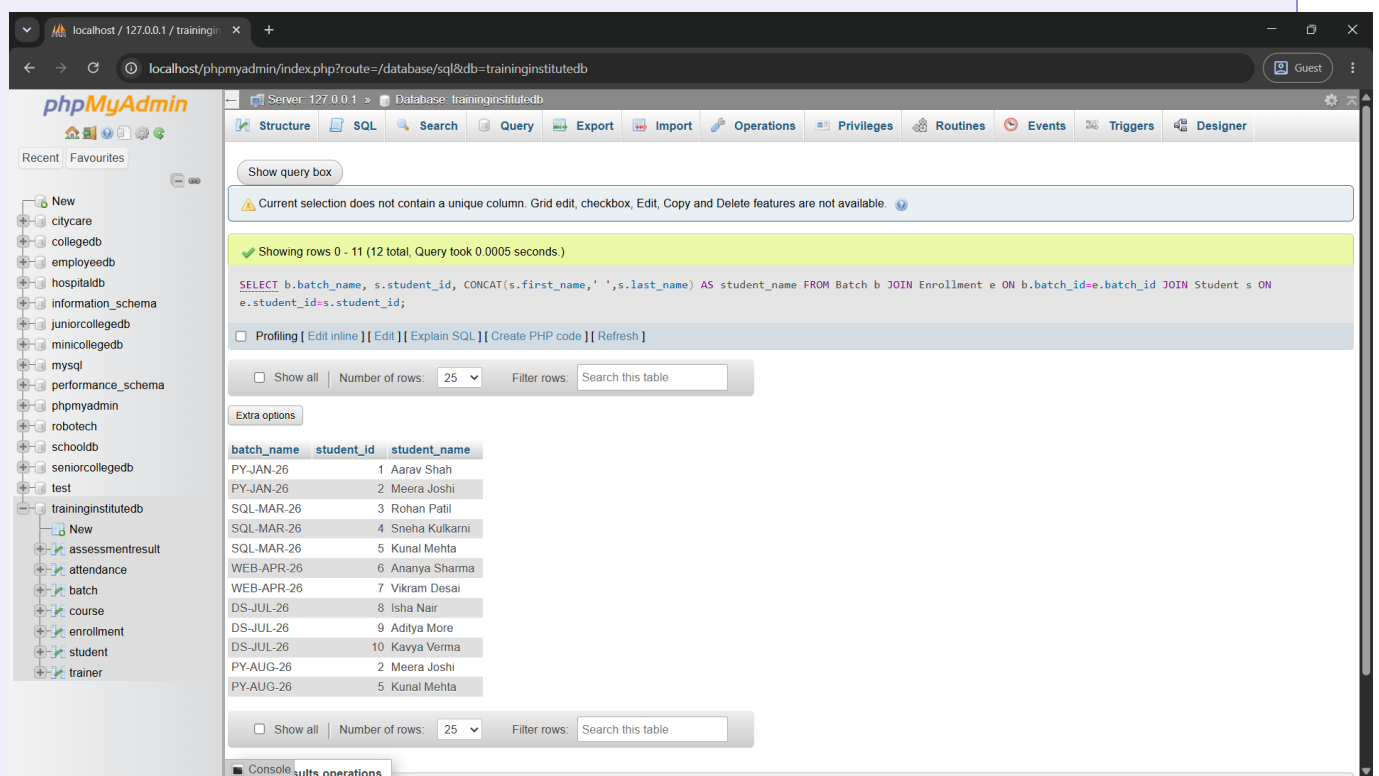
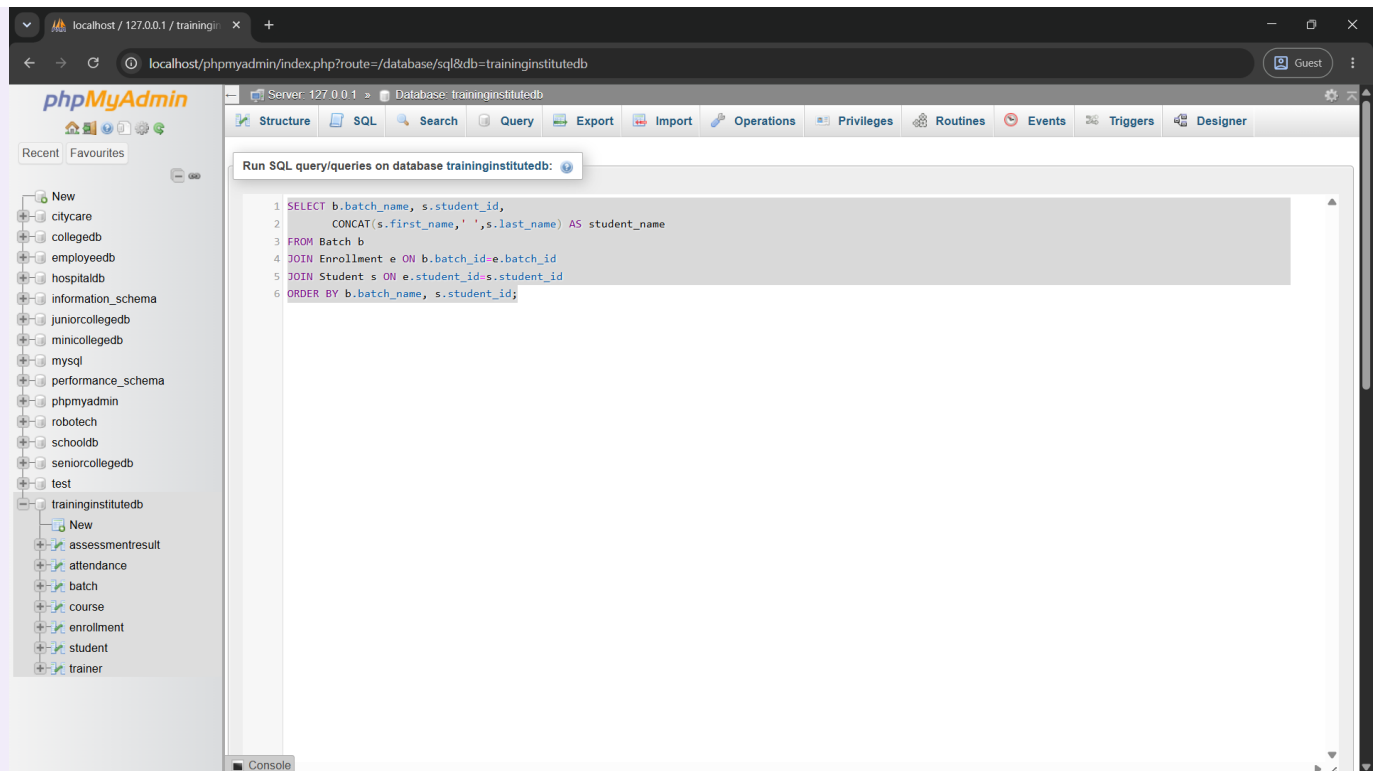
11. Business Queries and Data Analysis

Q1. Display all students enrolled in each batch.

```

SELECT b.batch_name, s.student_id,
       CONCAT(s.first_name, ' ', s.last_name) AS student_name
FROM Batch b
JOIN Enrollment e ON b.batch_id=e.batch_id
JOIN Student s ON e.student_id=s.student_id
ORDER BY b.batch_name, s.student_id;

```

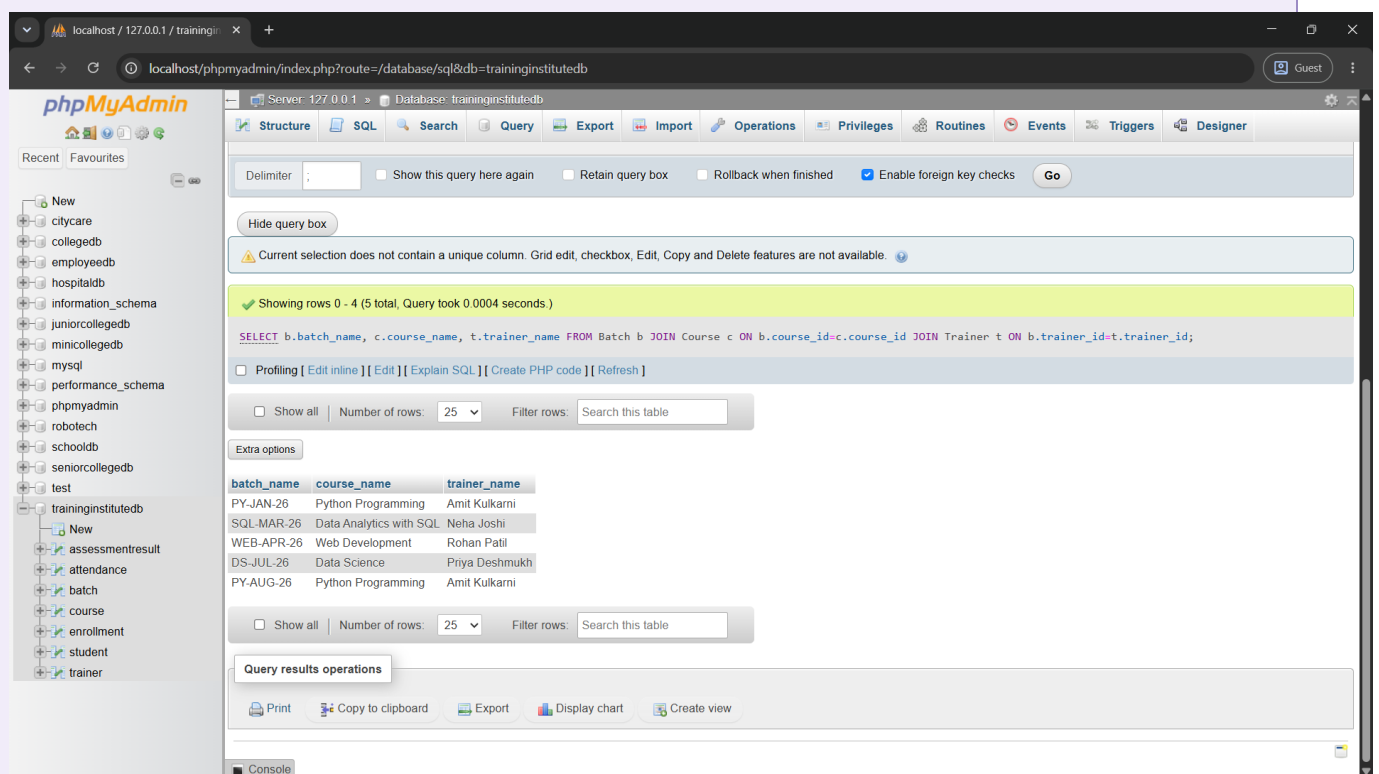
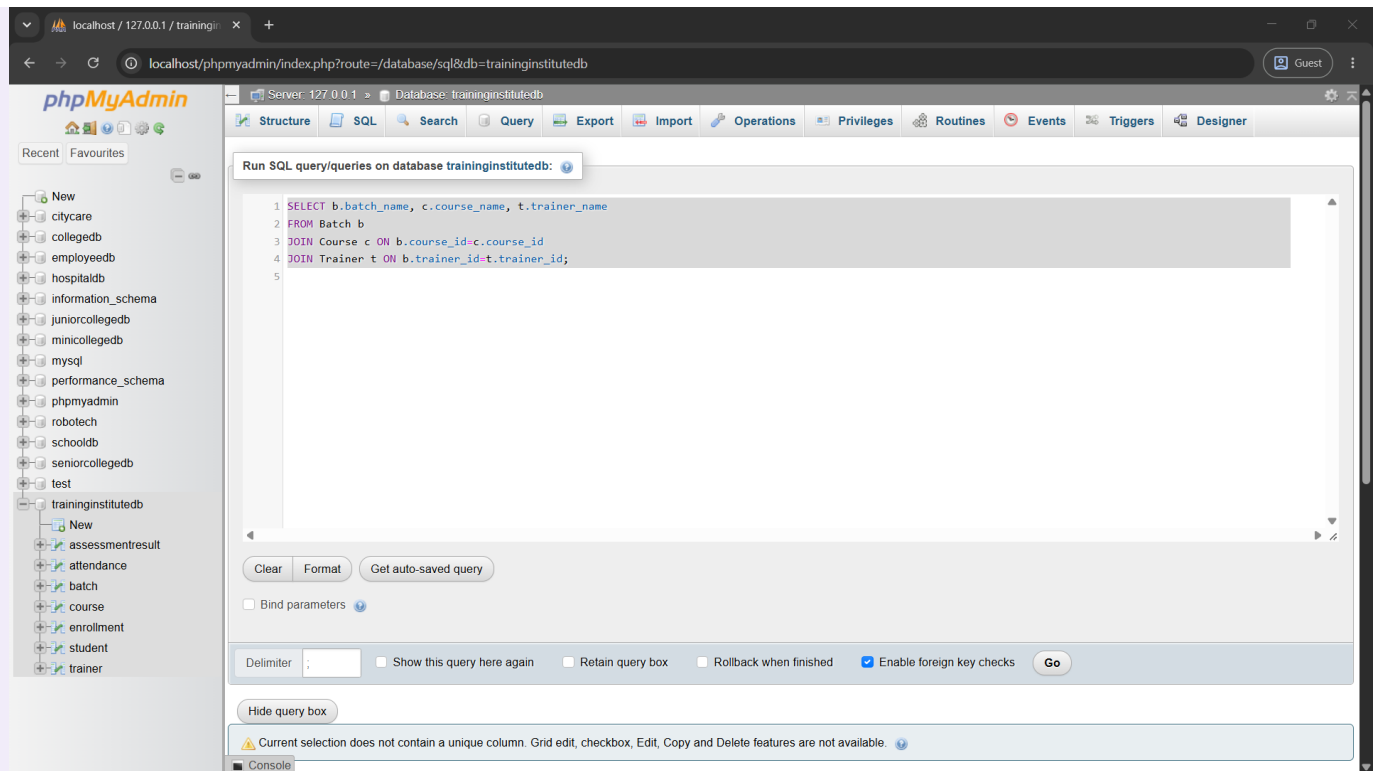


Q2. Display the course and trainer assigned to every batch.

```

SELECT b.batch_name, c.course_name, t.trainer_name
FROM Batch b
JOIN Course c ON b.course_id=c.course_id
JOIN Trainer t ON b.trainer_id=t.trainer_id;

```



Q3. Find the total number of students in each course.

```

SELECT c.course_name, COUNT(e.enrollment_id) AS total_students
FROM Course c
LEFT JOIN Batch b ON c.course_id=b.course_id
LEFT JOIN Enrollment e ON b.batch_id=e.batch_id
GROUP BY c.course_id, c.course_name

```

ORDER BY total_students DESC;

The screenshot shows the phpMyAdmin interface for the 'traininginstitutedb' database. The 'Table: Course' is selected. The SQL query editor contains the following query:

```
1 SELECT c.course_name, COUNT(e.enrollment_id) AS total_students
2 FROM Course c
3 LEFT JOIN Batch b ON c.course_id=b.course_id
4 LEFT JOIN Enrollment e ON b.batch_id=e.batch_id
5 GROUP BY c.course_id, c.course_name
6 ORDER BY total_students DESC;
7
```

The query is ready to be executed. The interface includes a sidebar with a database tree, a top navigation bar, and a bottom console area.

The screenshot shows the phpMyAdmin interface displaying the results of the SQL query. The query was successful, and the results are shown in a table with 4 rows. The table has two columns: 'course_name' and 'total_students'.

course_name	total_students
Python Programming	4
Data Analytics with SQL	3
Data Science	3
Web Development	2

The interface also shows the query execution details, including the number of rows (4) and the time taken (0.0012 seconds). The 'Query results operations' section includes links for Print, Copy to clipboard, Export, Display chart, and Create view.

Q4. Find batches having more than 2 students.

```
SELECT b.batch_name, COUNT(e.enrollment_id) AS total_students
FROM Batch b
JOIN Enrollment e ON b.batch_id=e.batch_id
GROUP BY b.batch_id, b.batch_name
```

```
HAVING COUNT(e.enrollment_id) > 2;
```

The screenshot shows the phpMyAdmin interface for the 'traininginstitutedb' database. The 'Batch' table is selected. The SQL query executed is: `SELECT b.batch_name, COUNT(e.enrollment_id) AS total_students FROM Batch b JOIN Enrollment e ON b.batch_id=e.batch_id GROUP BY b.batch_id, b.batch_name HAVING COUNT(e.enrollment_id) > 2;` The result shows two rows: 'SQL-MAR-26' with 3 students and 'DS-JUL-26' with 3 students. The interface includes a sidebar with database structure, a top navigation bar, and a bottom console.

batch_name	total_students
SQL-MAR-26	3
DS-JUL-26	3

Q5. Display students whose attendance is below 75%.

```
SELECT s.student_id,  
       CONCAT(s.first_name, ' ', s.last_name) AS student_name,  
       ROUND(100 * SUM(a.status = 'Present') / COUNT(a.attendance_id), 2) AS attendance_percentage  
FROM Student s  
JOIN Enrollment e ON s.student_id = e.student_id  
JOIN Attendance a ON e.enrollment_id = a.enrollment_id  
GROUP BY s.student_id, student_name  
HAVING attendance_percentage < 75;
```

localhost / 127.0.0.1 / trainingi... x +

localhost/phpmyadmin/index.php?route=/database/sql&db=traininginstitutedb

phpMyAdmin

Recent Favourites

New

- citycare
- collegedb
- employeeedb
- hospitaldb
- information_schema
- juniorcollegedb
- minicollegedb
- mysql
- performance_schema
- phpmyadmin
- robotech
- schooldb
- seniorcollegedb
- test
- traininginstitutedb

Database: traininginstitutedb > Table: Student

Browse Structure SQL Search Insert Export Import Privileges Operations Triggers

Run SQL query/queries on database traininginstitutedb:

```

1 SELECT s.student_id,
2       CONCAT(s.first_name, ' ', s.last_name) AS student_name,
3       ROUND(100 * SUM(a.status = 'Present') / COUNT(a.attendance_id), 2) AS attendance_percentage
4 FROM Student s
5 JOIN Enrollment e ON s.student_id = e.student_id
6 JOIN Attendance a ON e.enrollment_id = a.enrollment_id
7 GROUP BY s.student_id, student_name
8 HAVING attendance_percentage < 75;
9

```

Clear Format Get auto-saved query

☐ Bind parameters

Delimiter ; ☐ Show this query here again ☐ Retain query box ☐ Rollback when finished ☒ Enable foreign key checks Go

Hide query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Console

localhost / 127.0.0.1 / trainingi... x +

localhost/phpmyadmin/index.php?route=/database/sql&db=traininginstitutedb

phpMyAdmin

Recent Favourites

New

- citycare
- collegedb
- employeeedb
- hospitaldb
- information_schema
- juniorcollegedb
- minicollegedb
- mysql
- performance_schema
- phpmyadmin
- robotech
- schooldb
- seniorcollegedb
- test
- traininginstitutedb

Database: traininginstitutedb > Table: Student

Browse Structure SQL Search Insert Export Import Privileges Operations Triggers

Delimitter ; ☐ Show this query here again ☐ Retain query box ☐ Rollback when finished ☒ Enable foreign key checks Go

Hide query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Showing rows 0 - 2 (3 total, Query took 0.0012 seconds)

```

SELECT s.student_id, CONCAT(s.first_name, ' ', s.last_name) AS student_name, ROUND(100 * SUM(a.status = 'Present') / COUNT(a.attendance_id), 2) AS attendance_percentage FROM Student s JOIN Enrollment e ON s.student_id = e.student_id JOIN Attendance a ON e.enrollment_id = a.enrollment_id GROUP BY s.student_id, student_name HAVING attendance_percentage < 75;

```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

student_id	student_name	attendance_percentage
4	Sneha Kulkarni	60.00
5	Kunal Mehta	40.00
7	Vikram Desai	60.00

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

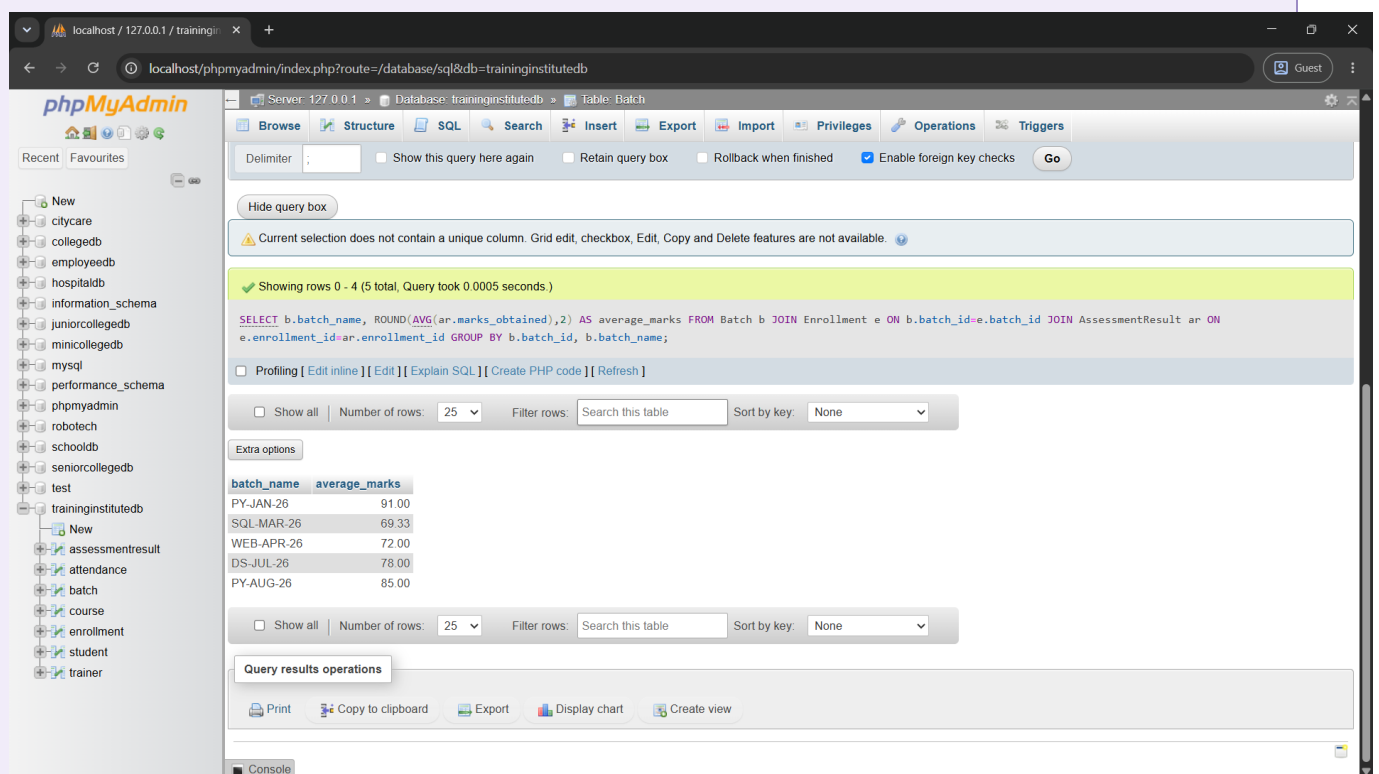
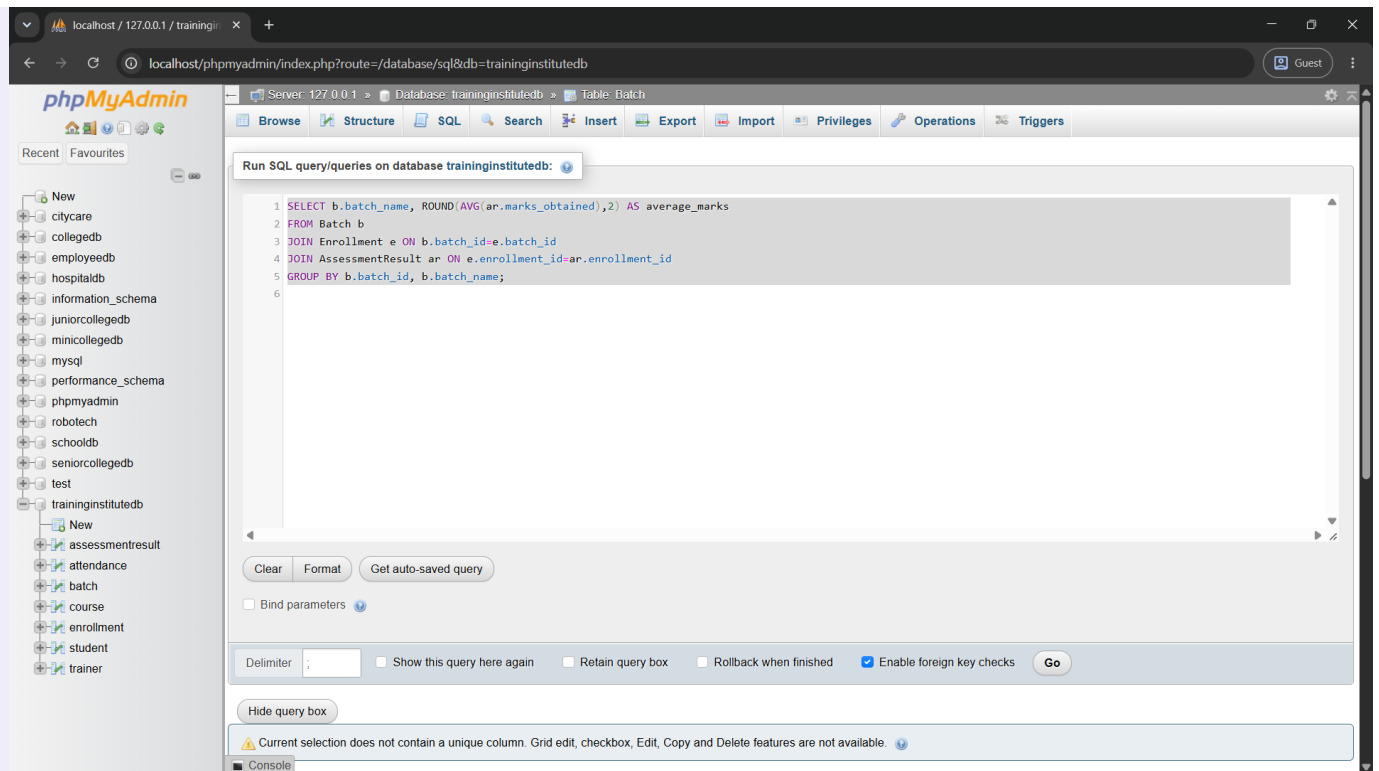
Console

Q6. Find the average assessment score for each batch.

```

SELECT b.batch_name, ROUND(AVG(ar.marks_obtained), 2) AS average_marks
FROM Batch b
JOIN Enrollment e ON b.batch_id = e.batch_id
JOIN AssessmentResult ar ON e.enrollment_id = ar.enrollment_id
GROUP BY b.batch_id, b.batch_name;

```



Q7. Identify the highest-performing student in every batch.

```

WITH RankedStudents AS (
  SELECT b.batch_name,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    ar.marks_obtained,
    RANK() OVER(PARTITION BY b.batch_id ORDER BY ar.marks_obtained DESC) AS rnk

```

```

FROM Batch b
JOIN Enrollment e ON b.batch_id=e.batch_id
JOIN Student s ON e.student_id=s.student_id
JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id
)
SELECT batch_name, student_name, marks_obtained
FROM RankedStudents
WHERE rnk=1;

```

Run SQL query/queries on database traininginstitutedb:

```

1 WITH RankedStudents AS (
2   SELECT b.batch_name,
3     CONCAT(s.first_name, ' ', s.last_name) AS student_name,
4     ar.marks_obtained,
5     RANK() OVER(PARTITION BY b.batch_id ORDER BY ar.marks_obtained DESC) AS rnk
6   FROM Batch b
7   JOIN Enrollment e ON b.batch_id=e.batch_id
8   JOIN Student s ON e.student_id=s.student_id
9   JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id
10 )
11 SELECT batch_name, student_name, marks_obtained
12 FROM RankedStudents
13 WHERE rnk=1;
14

```

Clear Format Get auto-saved query

☐ Bind parameters

Delimiter ; ☐ Show this query here again ☐ Retain query box ☐ Rollback when finished ☒ Enable foreign key checks Go

Hide query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Showing rows 0 - 4 (5 total, Query took 0.0025 seconds)

```

WITH RankedStudents AS ( SELECT b.batch_name, CONCAT(s.first_name, ' ', s.last_name) AS student_name, ar.marks_obtained, RANK() OVER(PARTITION BY b.batch_id ORDER BY ar.marks_obtained DESC) AS rnk FROM Batch b JOIN Enrollment e ON b.batch_id=e.batch_id JOIN Student s ON e.student_id=s.student_id JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id ) SELECT batch_name, student_name, marks_obtained FROM RankedStudents WHERE rnk=1;

```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

batch_name	student_name	marks_obtained
PY-JAN-26	Aarav Shah	94.00
SQL-MAR-26	Rohan Patil	91.00
WEB-APR-26	Ananya Sharma	76.00
DS-JUL-26	Isha Nair	83.00
PY-AUG-26	Meera Joshi	85.00

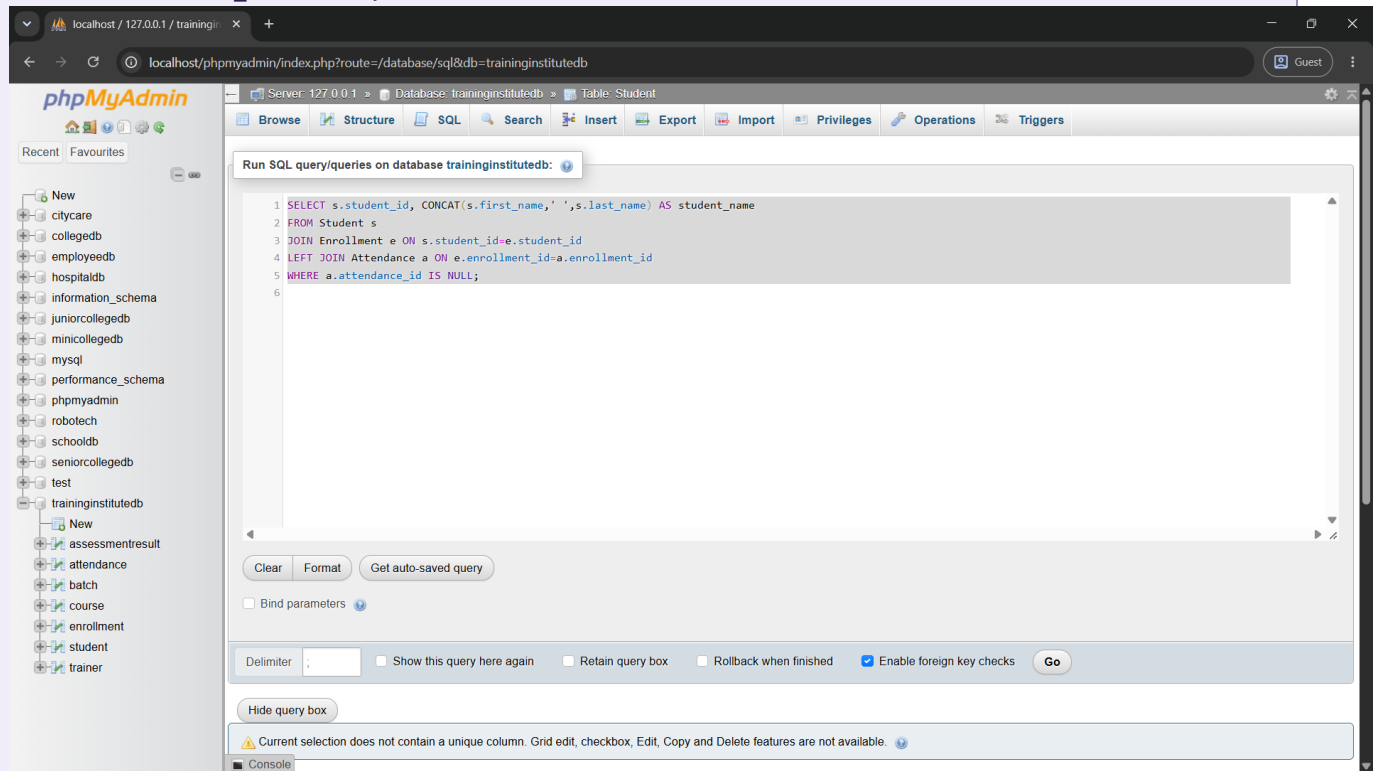
☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

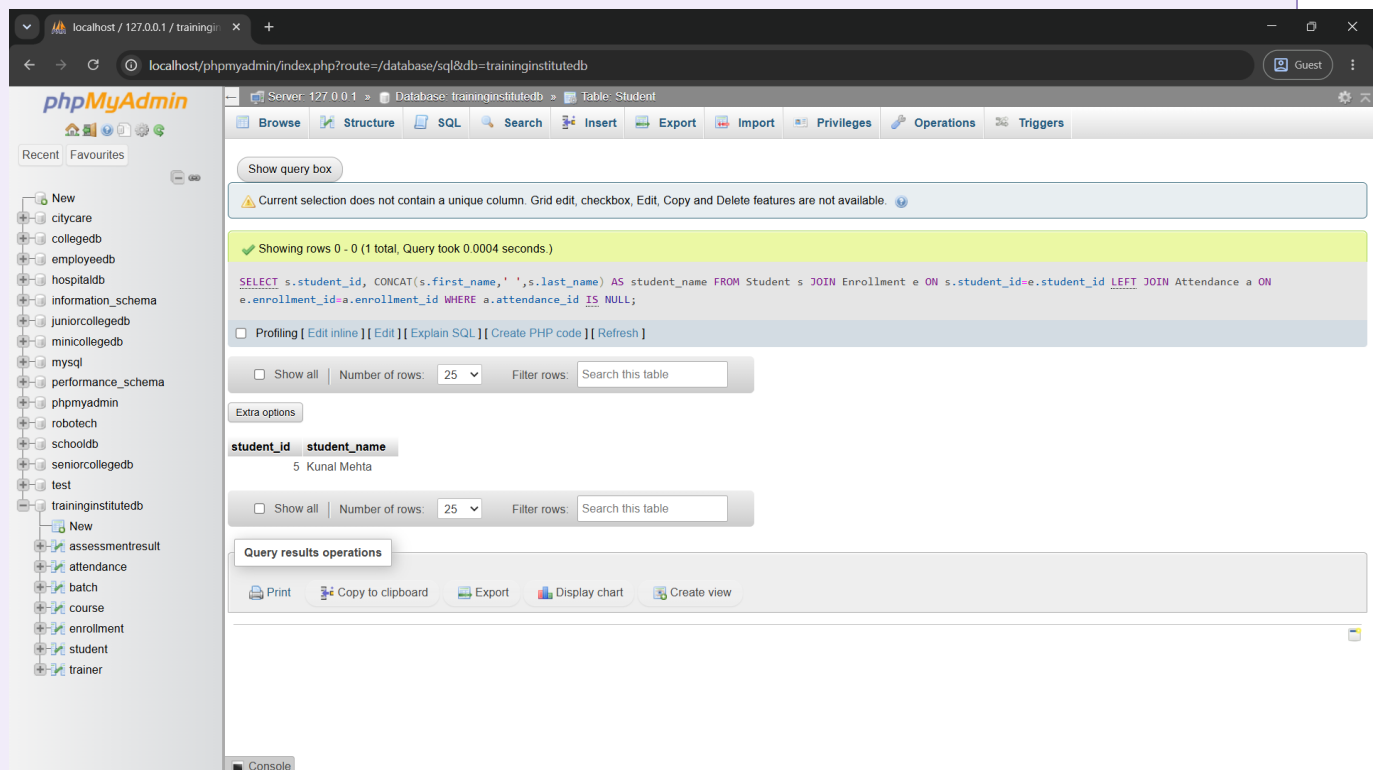
Print Copy to clipboard Create view

Q8. Display students who have not attended any session.

```
SELECT s.student_id, CONCAT(s.first_name, ' ', s.last_name) AS student_name
FROM Student s
JOIN Enrollment e ON s.student_id=e.student_id
LEFT JOIN Attendance a ON e.enrollment_id=a.enrollment_id
WHERE a.attendance_id IS NULL;
```



The screenshot shows the phpMyAdmin interface with the SQL editor open. The query is entered in the text area, and the 'Go' button is visible at the bottom right of the editor. A warning message at the bottom states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.'

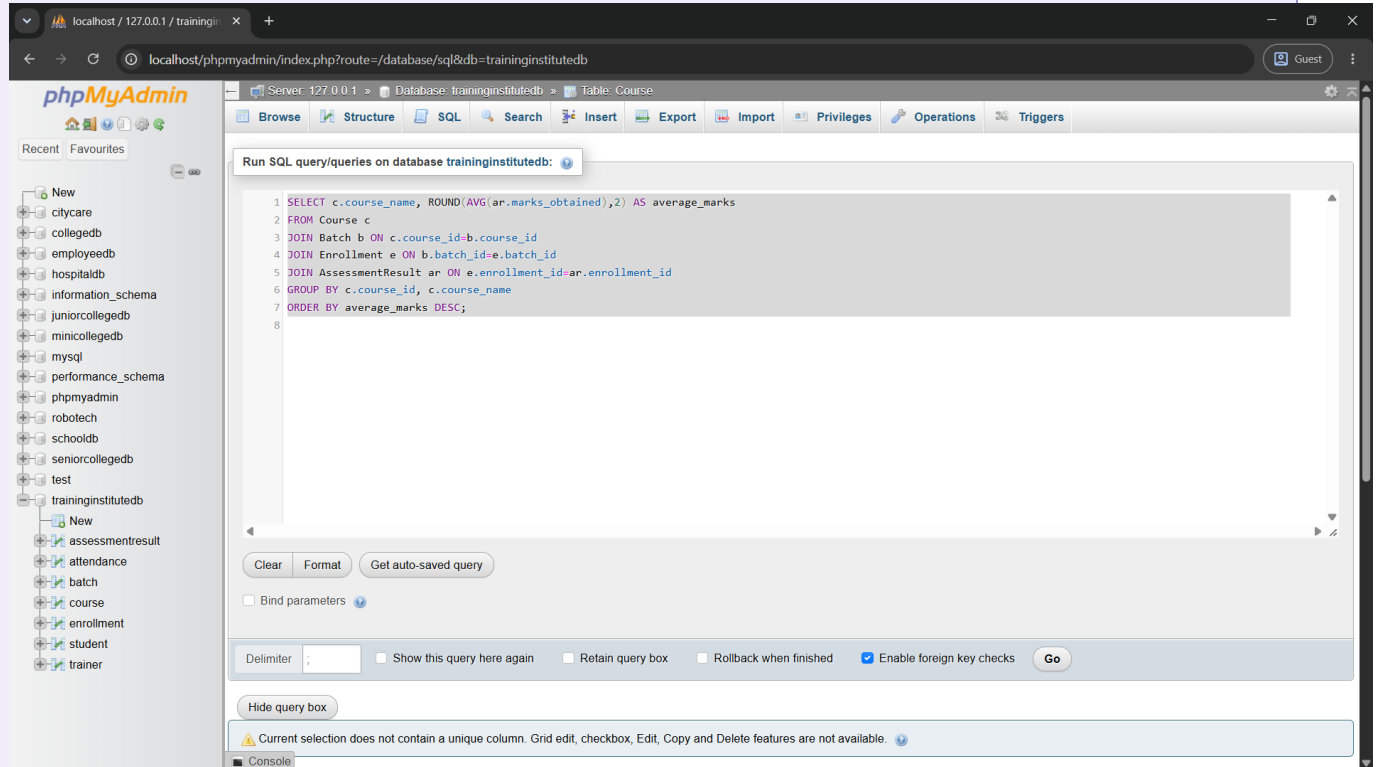


The screenshot shows the phpMyAdmin interface with the query results displayed. The results table has two columns: 'student_id' and 'student_name'. There is one row of data: student_id 5, student_name Kunal Mehta. The interface also shows the query execution status: 'Showing rows 0 - 0 (1 total, Query took 0.0004 seconds)'. Below the results, there are options for 'Query results operations' including Print, Copy to clipboard, Export, Display chart, and Create view.

student_id	student_name
5	Kunal Mehta

Q9. Compare course-wise student performance.

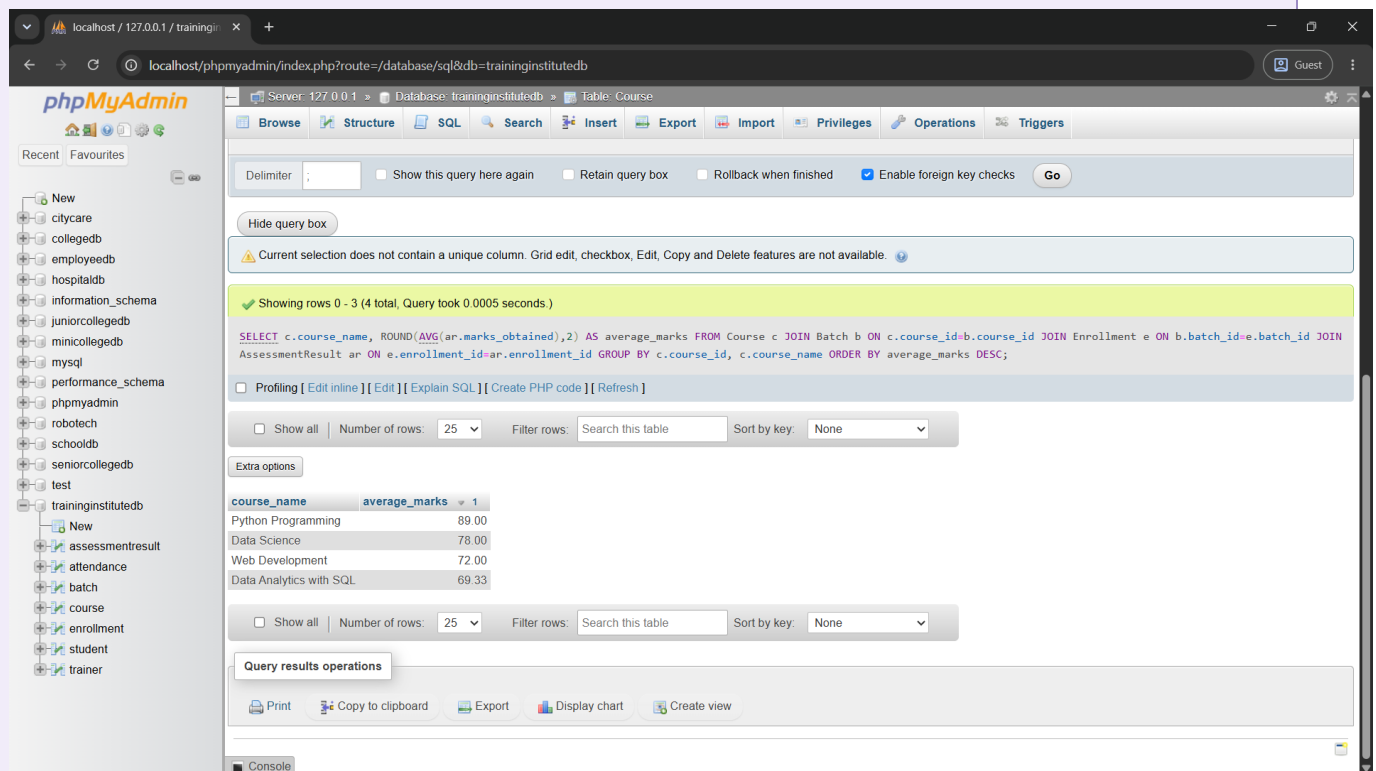
```
SELECT c.course_name, ROUND(AVG(ar.marks_obtained),2) AS average_marks
FROM Course c
JOIN Batch b ON c.course_id=b.course_id
JOIN Enrollment e ON b.batch_id=e.batch_id
JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id
GROUP BY c.course_id, c.course_name
ORDER BY average_marks DESC;
```



The screenshot shows the phpMyAdmin interface with the 'traininginstitutedb' database selected. The 'Run SQL query/queries' box contains the following SQL query:

```
1 SELECT c.course_name, ROUND(AVG(ar.marks_obtained),2) AS average_marks
2 FROM Course c
3 JOIN Batch b ON c.course_id=b.course_id
4 JOIN Enrollment e ON b.batch_id=e.batch_id
5 JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id
6 GROUP BY c.course_id, c.course_name
7 ORDER BY average_marks DESC;
8
```

Below the query box, there are buttons for 'Clear', 'Format', and 'Get auto-saved query'. There is also a checkbox for 'Bind parameters'. At the bottom, there are checkboxes for 'Show this query here again', 'Retain query box', 'Rollback when finished', and 'Enable foreign key checks', along with a 'Go' button.



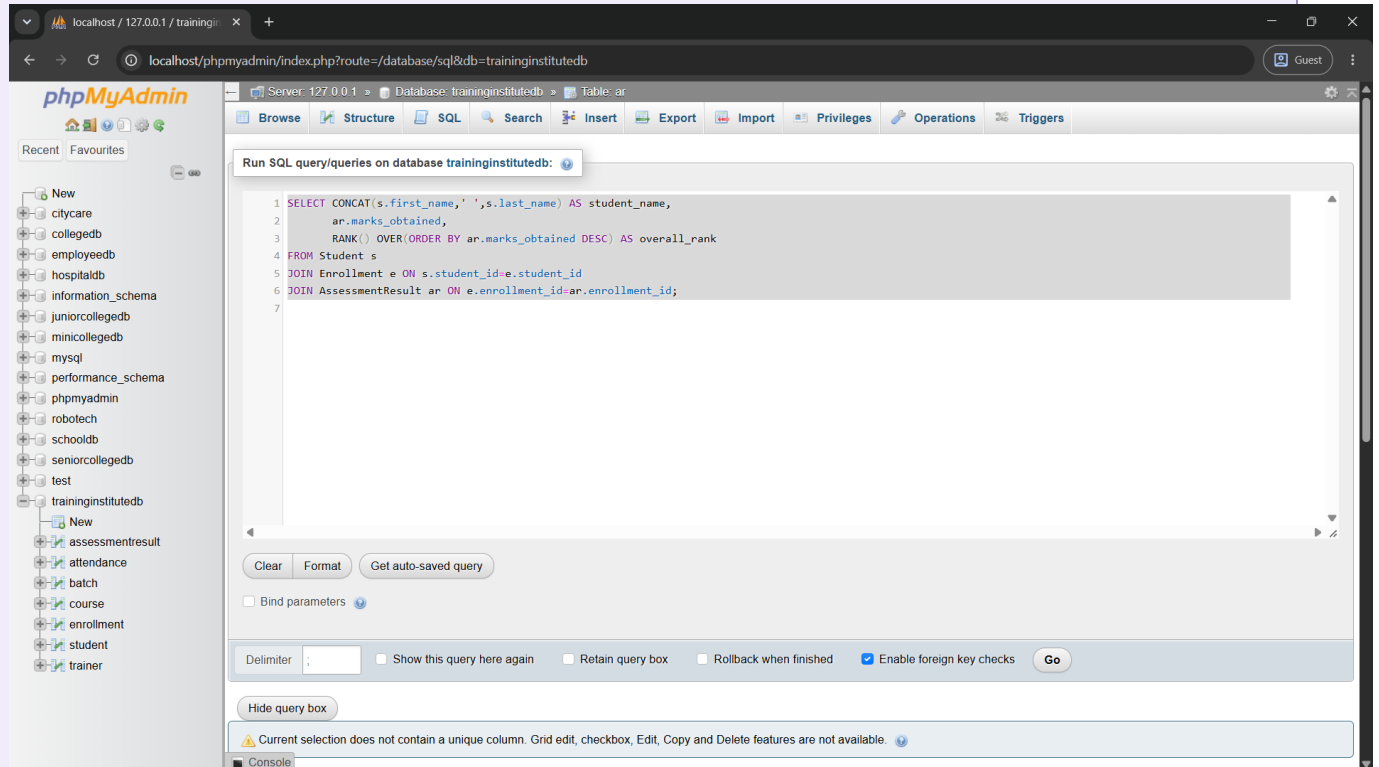
The screenshot shows the phpMyAdmin interface displaying the results of the SQL query. The results are shown in a table with two columns: 'course_name' and 'average_marks'. The table contains four rows of data:

course_name	average_marks
Python Programming	89.00
Data Science	78.00
Web Development	72.00
Data Analytics with SQL	69.33

Below the table, there are buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'. There is also a 'Query results operations' section with a 'Show all' button and a 'Number of rows' dropdown set to 25.

Q10. Rank students according to assessment results.

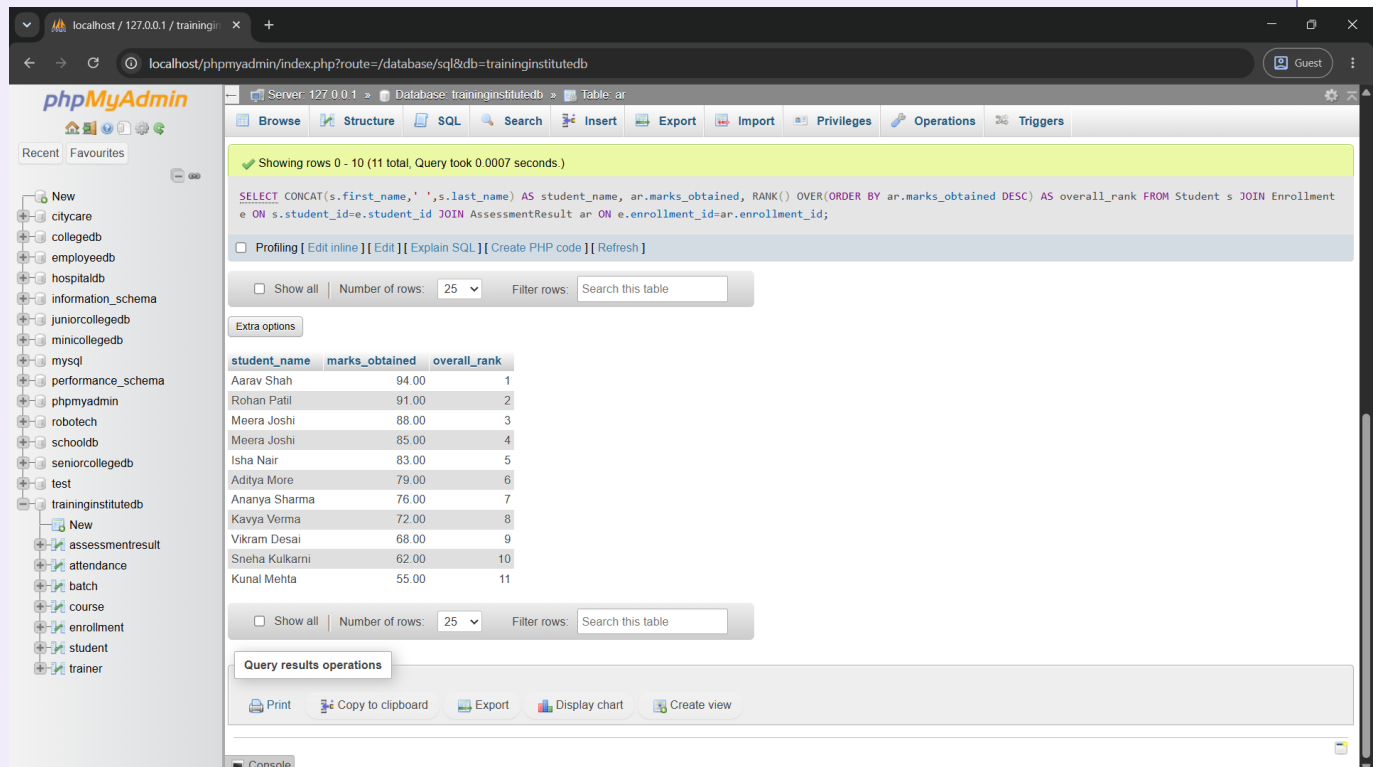
```
SELECT CONCAT(s.first_name, ' ', s.last_name) AS student_name,  
       ar.marks_obtained,  
       RANK() OVER(ORDER BY ar.marks_obtained DESC) AS overall_rank  
FROM Student s  
JOIN Enrollment e ON s.student_id=e.student_id  
JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id;
```



The screenshot shows the phpMyAdmin interface with the 'traininginstitutedb' database selected. The SQL query is entered in the 'Run SQL query/queries' box. The query is as follows:

```
1 SELECT CONCAT(s.first_name, ' ', s.last_name) AS student_name,  
2       ar.marks_obtained,  
3       RANK() OVER(ORDER BY ar.marks_obtained DESC) AS overall_rank  
4 FROM Student s  
5 JOIN Enrollment e ON s.student_id=e.student_id  
6 JOIN AssessmentResult ar ON e.enrollment_id=ar.enrollment_id;  
7
```

Below the query box, there are buttons for 'Clear', 'Format', and 'Get auto-saved query'. There is also a checkbox for 'Bind parameters'. At the bottom, there are checkboxes for 'Show this query here again', 'Retain query box', 'Rollback when finished', and 'Enable foreign key checks', along with a 'Go' button. A warning message at the bottom states: 'Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.'



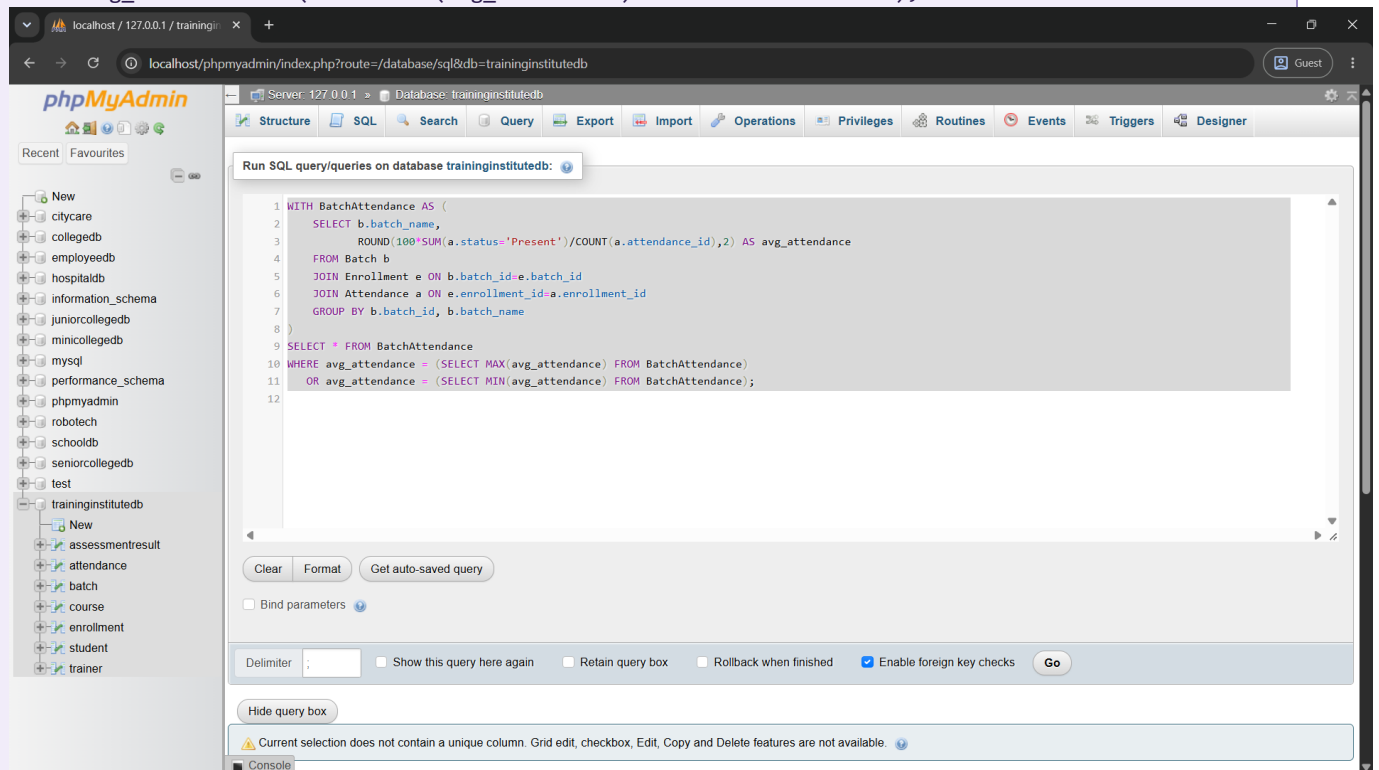
The screenshot shows the phpMyAdmin interface displaying the results of the SQL query. The results are shown in a table with the following columns: student_name, marks_obtained, and overall_rank. The table contains 11 rows of data.

student_name	marks_obtained	overall_rank
Aarav Shah	94.00	1
Rohan Patil	91.00	2
Meera Joshi	88.00	3
Meera Joshi	85.00	4
Isha Nair	83.00	5
Aditya More	79.00	6
Ananya Sharma	76.00	7
Kavya Verma	72.00	8
Vikram Desai	68.00	9
Sneha Kulkarni	62.00	10
Kunal Mehta	55.00	11

Below the table, there are buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'. There is also a 'Query results operations' section.

Q11. Find batches with the highest and lowest average attendance.

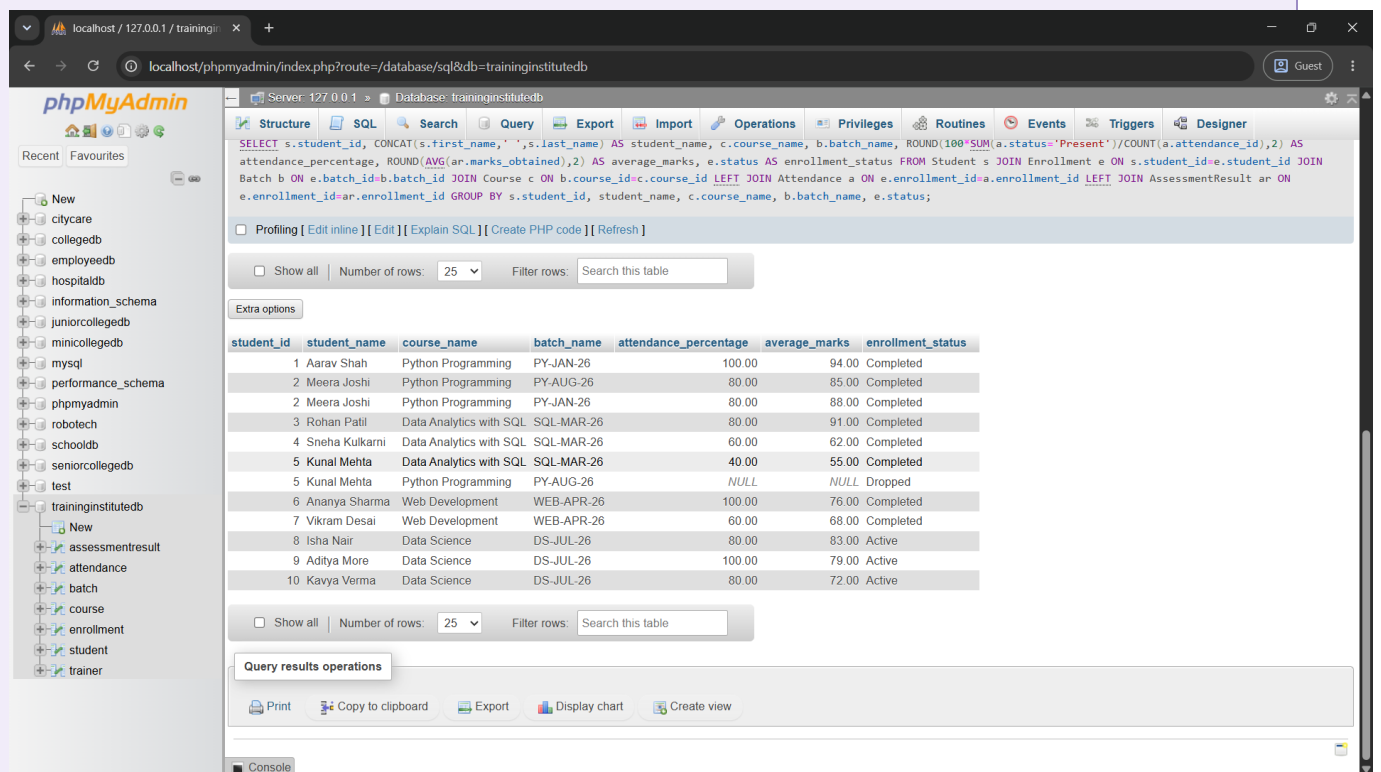
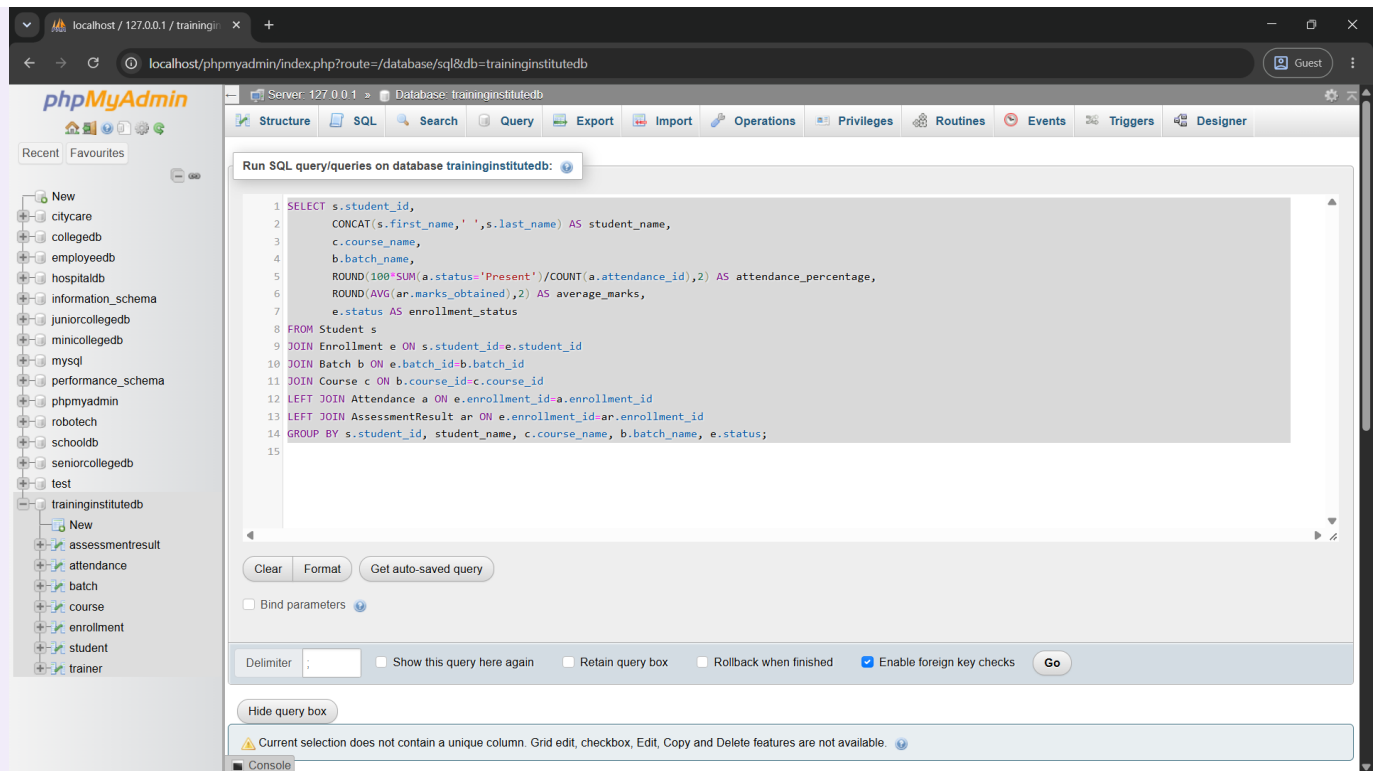
```
WITH BatchAttendance AS (  
    SELECT b.batch_name,  
           ROUND(100*SUM(a.status='Present')/COUNT(a.attendance_id),2) AS avg_attendance  
    FROM Batch b  
    JOIN Enrollment e ON b.batch_id=e.batch_id  
    JOIN Attendance a ON e.enrollment_id=a.enrollment_id  
    GROUP BY b.batch_id, b.batch_name  
)  
SELECT * FROM BatchAttendance  
WHERE avg_attendance = (SELECT MAX(avg_attendance) FROM BatchAttendance)  
OR avg_attendance = (SELECT MIN(avg_attendance) FROM BatchAttendance);
```



The screenshot shows the phpMyAdmin web interface. The left sidebar lists various databases, with 'traininginstitutedb' selected. The main panel displays a complex SQL query in the 'SQL' tab. The query is designed to calculate average attendance and average marks for students across different batches and courses. Below the query, the results are shown in a table with two columns: 'batch_name' and 'avg_attendance'. The results show two batches: 'PY-JAN-26' with an average attendance of 90.00, and 'SQL-MAR-26' with an average attendance of 60.00. The interface also includes a 'Query results operations' section with buttons for 'Print', 'Copy to clipboard', and 'Create view'.

Q12. Generate a student-wise training progress report.

```
SELECT s.student_id,
       CONCAT(s.first_name, ' ', s.last_name) AS student_name,
       c.course_name,
       b.batch_name,
       ROUND(100 * SUM(a.status = 'Present') / COUNT(a.attendance_id), 2) AS attendance_percentage,
       ROUND(AVG(ar.marks_obtained), 2) AS average_marks,
       e.status AS enrollment_status
FROM Student s
JOIN Enrollment e ON s.student_id = e.student_id
JOIN Batch b ON e.batch_id = b.batch_id
JOIN Course c ON b.course_id = c.course_id
LEFT JOIN Attendance a ON e.enrollment_id = a.enrollment_id
LEFT JOIN AssessmentResult ar ON e.enrollment_id = ar.enrollment_id
GROUP BY s.student_id, student_name, c.course_name, b.batch_name, e.status;
```



12. Expected Analysis from the Sample Data

Analysis Area	Finding
Course enrollment	Python Programming has 4 enrollments across two batches; Data Analytics with SQL has 3; Web Development has 2; Data Science has 3.

Low attendance	The sample attendance records include students below the 75% threshold, which can be identified using Q5.
Batch performance	Assessment records allow average marks to be calculated and compared batch-wise.
High performers	Scores such as 94, 91 and 88 can be identified through assessment ranking queries.
Training progress	The progress report combines course, batch, attendance, assessment performance, and enrollment status.

13. How the Institute Can Use the Results

- Use enrollment counts to understand demand for each course and batch.
- Use attendance percentages to identify students who may need follow-up.
- Use assessment averages and rankings to identify strong performance and students who may need additional support.
- Use trainer and batch information to monitor course delivery.
- Use the student-wise progress report to support academic and training decisions.

14. Required SQL Techniques Demonstrated

Technique	Use in this Project
Joins	Connect Student, Enrollment, Batch, Course, Trainer, Attendance and AssessmentResult.
Aggregate functions	COUNT() and AVG() summarize enrollments and performance.
Subqueries	Compare batch attendance with maximum and minimum values.
Views	Can be used for frequently requested reports such as student-wise progress.
Window functions	RANK() is used for batch-wise and overall student ranking.

15. Conclusion

The Training Institute Management System provides a structured relational database for managing students, trainers, courses, batches, enrollments, attendance, and assessment results. The ER model establishes the required one-to-many relationships and foreign keys maintain referential integrity. The business queries demonstrate how MySQL can convert operational records into useful reports about enrollment, attendance, and student performance.

16. Submission Checklist

- ☐ ER diagram showing all seven required tables and relationships.
- ☐ Primary key for every table.
- ☐ Foreign keys for all required relationships.
- ☐ MySQL database and table creation script.
- ☐ Realistic sample data in all tables.
- ☐ Business queries using joins, aggregates, subqueries and window functions where suitable.
- ☐ Analysis, insights, and conclusion.

☐ Separate MySQL script file included with this report.

Prepared according to Assignment No. 12 of the supplied DBMS workbook.

17. Final Relational Schema Summary

COURSE(course_id PK, course_name, duration_weeks, fee, course_level)

TRAINER(trainer_id PK, trainer_name, email, specialization, experience_years)

BATCH(batch_id PK, batch_name, course_id FK, trainer_id FK, start_date, end_date, timing, max_capacity)

STUDENT(student_id PK, first_name, last_name, email, phone, city, registration_date)

ENROLLMENT(enrollment_id PK, student_id FK, batch_id FK, enrollment_date, status)

ATTENDANCE(attendance_id PK, enrollment_id FK, session_date, status)

ASSESSMENT_RESULT(result_id PK, enrollment_id FK, assessment_name, assessment_date, marks_obtained, max_marks)

The relational schema is consistent with the ER model: every strong entity has its own primary key, and every required one-to-many relationship is implemented through a foreign key on the many-side table.